Slide 1

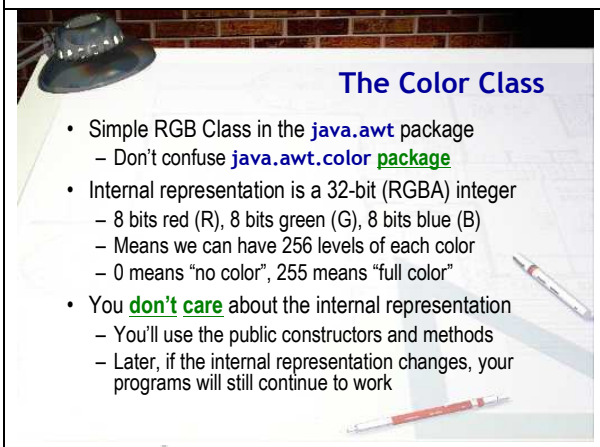
CS 170
Java Programming 1
Local Color
Duration: 00:01:06
Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture, Local Color. This lecture is kind of a last-minute addition. I had planned on covering this material in class before the midterm, but we didn't get to it, so I've added it here.

In addition, some of the exercises this week build upon the Artist applet we started in Lecture last week. Since we didn't complete that either, I've provided a link to a completed version you can work with. You'll find it just under the link for this lecture. Go ahead and open BlueJ and start a new project (ic10). Then download the file from the Web page and store it in the ic10 folder that BlueJ created. (That way it won't overwrite the Artist.java file in your ic09 project.) As usual, you may need to close the project and re-open it for the new class to appear.

Now, let's take a look at the Color class.



Slide 2

The Color Class
Duration: 00:02:16
Advance mode: Auto

Notes:

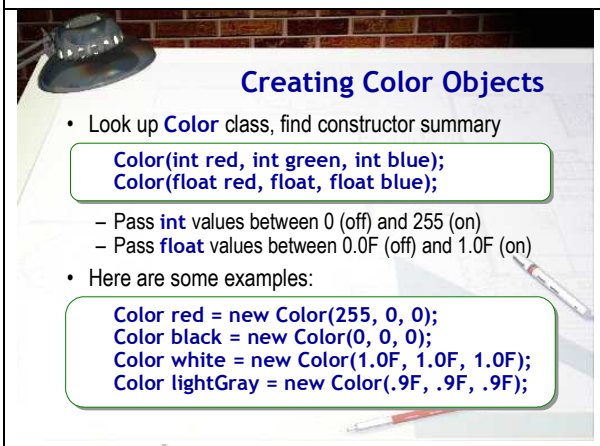
To color your Artist's cottage we'll use the Color class located in the java.awt package. If you've already imported java.awt.* at the top of your program (like you will for most applets), then the Color class will already be included. If you go looking through the docs for the Color class, though, don't be confused by the java.awt.color package which is used for high-end Photoshop-type color manipulation.

Internally, Color objects in Java are contain a single field which is a 32-bit integer. Rather than using that number directly, though the class breaks the number into three or four pieces.

Logically, the Color class uses three 8-bit numbers to represent the portion of red, green, and blue that make up a color on your computer. This is called RGB color; knowing this will help you remember that red goes first, green second, and blue last. 8-bits are also used for the so-called "alpha channel" which controls the opacity and transparency of the color.

Since eight bits are used to represent each portion of the color, you know that we can have 256 levels of each color. If the red portion of the number is set to 0, that means that there is no red in the result. If the red portion is set to 255, then that means the maximum amount of red. The same thing is true of the other two colors. For the alpha channel, 0 means that the color is completely transparent (and thus invisible) while 255 means that it is opaque, so no color "bleeds through" from underneath. Most of the time you can just ignore the alpha channel and you'll get regular opaque colors.

You, however, don't really worry about the internal representation; instead, you'll use the public constructors and methods to create and manipulate Color objects. Then, if later

	Sun decides to change the internal structure of Color objects to use 64-bit longs, doubles or floats, you really won't care at all. That's the beauty of encapsulation.
 **Slide 3** **Creating Color Objects** Duration: 00:01:59 Advance mode: Auto	**Notes:** Now, go ahead and look up the JavaDocs for the Color class. When you've found it, look up the constructor summary and locate these two constructors. These are the two we'll want to work with first. The first constructor takes three int values. If you pass 0 for one of the values, that color is "turned off". If you pass 255, then the primary color is represented at full intensity. Values in between vary between completely on and completely absent. The only problem with the int constructor is that sometimes it's mentally a little difficult to figure out what numbers to use for each color. Many of us find it easier to deal with percentages: 10% red, 90% blue and so on. That's what the second constructor I've shown here is for. To specify 10% red, you'd pass .1F and for 90% blue, you'd specify .9F. Remember, you do have to add the F on the end of the values you pass, so that it knows to use this version of the constructor. Here are some examples so you can see how this works: • This first example creates a red Color object using the three-int constructor, since we set the red parameter to 255 and the other two colors to 0. • The second example turns off all three colors, so we get black. (If we wanted white, we'd set all three values to 255 using the int constructor.) • Of course, if we want white using the float constructor, we just pass 1.0F for each parameter as in this third example. • Finally, here's a 90% gray. Any time you supply the same value for each color, the result will be some form of gray.

Slide 4

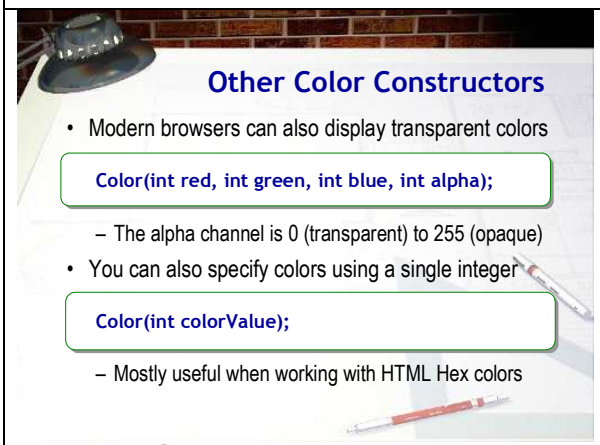
The setColor() Method

Duration: 00:00:55
Advance mode: Auto

Notes:

Once you've created a Color object, you can use it in several ways. One of the easiest ways is to pass it to a Graphics object's setColor() method, like the example shown here. When you change the painting color using setColor(), then any subsequent drawing statements in the paint() method will use the new color, not just the next drawing statement. If you want to revert to the previous color, you'll need to use setColor again.

OK, now let's see if you can put this to work. If you haven't already created an IC exercise document this week, start up a new one and create a section for this lecture. Then, for Exercise 1, modify the Artist applet to put green windows and a red door on the Artist's cottage. When you're done, run it and shoot me a screen-shot.



Slide 5

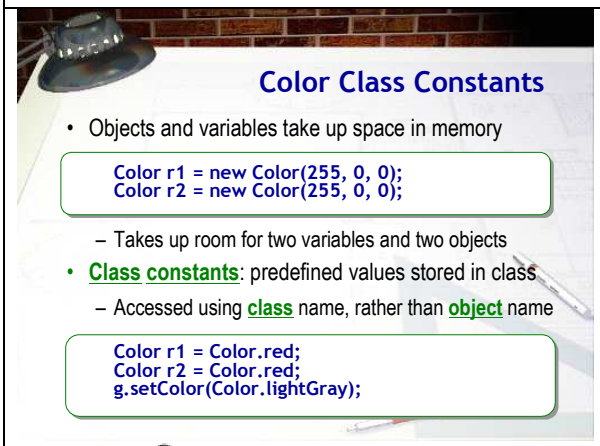
Other Color Constructors

Duration: 00:01:10
Advance mode: Auto

Notes:

Now, let's take a look at some of the other Color constructors. Almost all modern browsers can now display transparent colors. To create a transparent color, you use one of the four-parameter constructors. Here's the int version, but the float version is similar. When you use one of these constructors, passing 0 makes the color completely transparent (which is kind of self-defeating, since it's then invisible), while passing 255 makes it completely opaque (which is also kind of useless, since you could have just used the three-parameter constructor to get the same result.)

There is also a constructor that allows you to supply a single integer value that looks like this. You'll normally use this constructor when trying to match a hexadecimal color value embedded into a Web page. By using the single parameter constructor, (and writing your color value in hexadecimal format), you avoid the mental translation of breaking the number down into its red-green-and-blue portions.

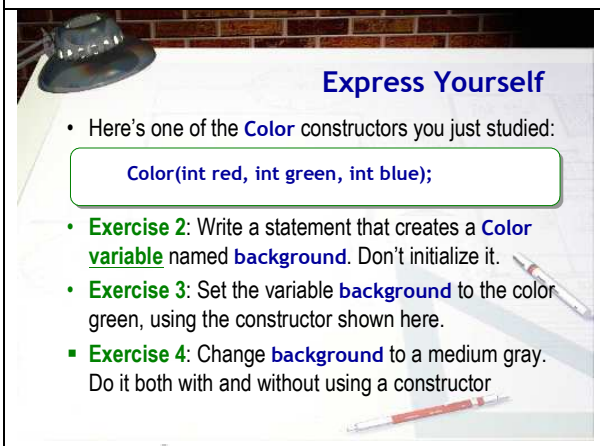


Slide 6

Notes:

Every time you create an object, both the object variable and the object it refers to take up some space in memory. Consider, for instance, these two Color variables and associated objects. In memory, your program will have to reserve space for two variables and the actual values that make up the colors. Note that in this case, though, the Color objects both contain exactly the same value: the color red. Wouldn't it be nice if we could store the "red" Color object once in memory and just "point" to it every time it was used?

That's where the Color class constants come in. These constants are predefined Color objects, of the sixteen or so most commonly used colors, that are then stored inside the class itself. To access these objects, you can use the name of

Color Class Constants Duration: 00:01:21 Advance mode: Auto	the class (Color in this case), along with the constant name. The constants have names like red, green and so on, as you can see here. There are actually two names for each color constant: one in upper case and one in lower case. The upper-case names don't work on older versions of Java, but you can use either one if you're using Java 5 or Java 6.
 **Slide 7** **Express Yourself** Duration: 00:01:06 Advance mode: Auto	**Notes:** OK, let's finish up this mini-lecture with some Code Pad exercises. When you've done all three of these, shoot me a screen-shot of the Code Pad window. You don't need a screen-shot for each one. Here's one of the Color constructors you just studied. • For Exercise 2 write a statement in Code Pad that creates a Color variable named background. Do not initialize it. • For Exercise 3, set the variable background to the color green. Use the constructor shown here. • Finally, for Exercise 4, change background to a medium gray. Do it twice. Use this constructor the first time. Do it a second time without using a constructor at all. Well, that's it. In the next lecture you'll make your Artist cottage a little more flexible by introducing variables and a little more ornate by using fonts.